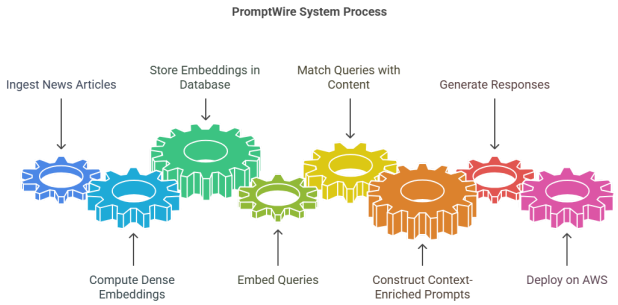
PromptWire: A RAG Approach Leveraging News Content

A Serverless RAG System for Real-Time, Context-Aware Question Answering

Swaraj Bhanja, Patsachon Pattakulpong
Department of Data Science and Artificial Intelligence
Asian Institute of Technology
Pathumthani, Thailand
{st125052, st124952}@ait.asia

ABSTRACT

Despite the advancement of large language models (LLMs) in 2025, these models remain limited in their ability to incorporate evolving real-world knowledge without retraining. To address this, we developed PromptWire, a modular retrieval-augmented generation (RAG) system designed to ground LLM outputs in timely, factual context using recent news data. PromptWire continuously ingests articles from public news APIs, computes unique identifiers, and logs article metadata into DynamoDB to prevent duplicate processing. Embeddings are generated using OpenAI's models via a Flask-based microservice hosted on Google Cloud Platform (GCP) and stored in a Weaviate vector database optimized for semantic search. Queries are routed through a JWT-secured Flask backend, which forwards them to a GCP-hosted retrieval API that performs semantic search, prompt construction, and GPT-4-based response generation. The system is orchestrated using a hybrid architecture: AWS handles ingestion and periodic health monitoring via Lambda, Kinesis, and EventBridge, while GCP manages retrieval, embedding, and generation. PromptWire offers a cost-efficient and adaptable approach for grounding LLMs in current information by avoiding static datasets and model fine-tuning. This setup is a reproducible template for deploying LLM pipelines in high-change domains such as journalism, finance, and public policy, where recency and contextual relevance are critical.



CCS CONCEPTS

Retrieval models and ranking~Document representation~Natural language generation~Similarity and distance measures~Cloud computing~Software architectures

KEYWORDS

Retrieval-Augmented Generation (RAG); Large Language Models (LLMs); Semantic Search; OpenAI; Amazon Web Services (AWS); Serverless Architecture; DynamoDB; GPT-4; News Ingestion; Context-Aware Generation; Information Retrieval; Vector Embeddings; Low-Cost Cloud Deployment; AI-Powered Question Answering;

ACM Reference format:
Swaraj Bhanja, Patsachon Pattakulpong. 2025. PromptWire: A RAG Approach Leveraging News Content

1. INTRODUCTION

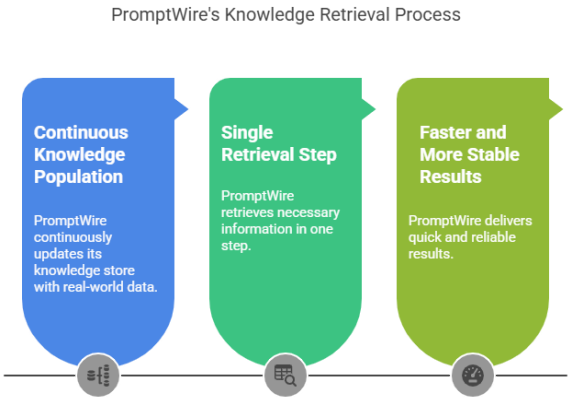
In the rapidly evolving landscape of artificial intelligence, people increasingly depend on intelligent systems to make sense of the world around them. The information ecosystem is in constant flux, from global economic shifts and political developments to scientific breakthroughs and humanitarian crises. At the same time, user expectations for digital assistants and language technologies have risen sharply. People no longer want answers that are simply fluent; they want answers that are informed, up to date, and relevant to unfolding events.

Large language models (LLMs), such as OpenAI's GPT-4 and the recent GPT-4o, have significantly advanced natural language processing and multimodal reasoning. GPT-4o, introduced in 2024, integrates capabilities across text, audio, and vision, enabling fluid interaction and more profound understanding across diverse input modalities. As part of these developments, OpenAI also rolled out built-in internet retrieval functionality, allowing ChatGPT to access and cite real-time information. This marked a significant shift, not only in what LLMs can do, but in how they interact with external knowledge. Despite these

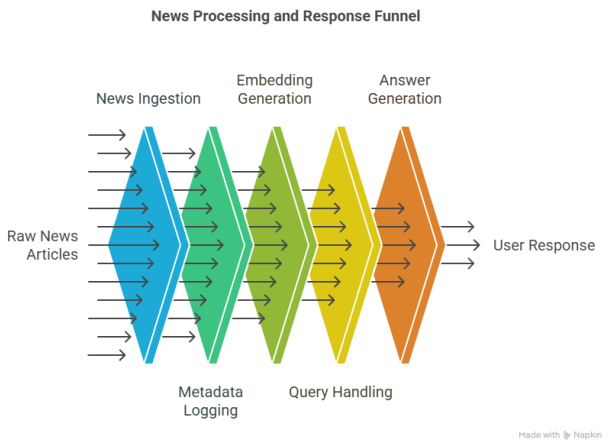
breakthroughs, however, most LLMs deployed in research or production do not come with native, always-on web browsing or up-to-date corpora. Their core models still rely on training data that quickly goes out of date, often by deployment time. This results in significant limitations when users query them about current events, recent policy changes, or market-specific news. In such cases, the model fails not due to a lack of reasoning ability, but due to a lack of access to the present.

This limitation presents an opportunity: what if we changed its context instead of modifying the model? Retrieval-Augmented Generation (RAG) is a paradigm that augments an LLM's generative capabilities by injecting it with relevant, retrieved information at inference time. Rather than relying solely on what is "in the model," RAG systems connect to external knowledge stores, select contextually relevant documents, and feed them into the model's prompt to ground its response. This improves the factual alignment of outputs and allows systems to evolve with the knowledge they retrieve. Several prominent RAG architectures have demonstrated this principle effectively. Facebook AI's RAG framework (Lewis et al., 2020) and Google's REALM (Guu et al., 2020) were early breakthroughs, combining dense vector retrieval with generative language models. However, these approaches rely on static corpora such as Wikipedia or Common Crawl, making them ill-suited for high-change domains like finance or breaking news. More recently, web browsing enabled LLMs have attempted to close this gap by performing live internet searches at inference time, but this introduces additional latency, inconsistent results, and architectural complexity.

To address these challenges, we introduce PromptWire, a modular and cloud-native retrieval-augmented generation system designed to ground LLM outputs in real-time information from the latest news. PromptWire continuously ingests news articles from public APIs and processes them using OpenAI's embedding models to produce dense semantic vectors. These vectors and metadata, such as titles, timestamps, and sources, are stored in Weaviate, an open-source vector database designed for semantic and hybrid search. When a user submits a query, the system generates an embedding for the question, performs an approximate nearest neighbor search in Weaviate to identify relevant articles, and injects the retrieved content into a prompt sent to GPT-4.1 for response generation. Unlike browser-based LLMs like GPT-4o, which conditionally invoke a retrieval step only after determining their insufficient internal knowledge or being explicitly prompted to, PromptWire continuously populates its knowledge store in advance. This eliminates the need for two-phase instruction processing and enables the model to operate with a single retrieval step, delivering faster and more stable results grounded in curated real-world data.



PromptWire's architecture is fully serverless and modular, relying on AWS Lambda for computation and AWS EventBridge for automation. This design enables low-cost, low-maintenance deployment suitable for resource-constrained environments such as academic labs or startups. While PromptWire is not a search engine, it bridges a critical gap between the generative capabilities of modern LLMs and the need for accurate, time-sensitive information retrieval. Leveraging serverless architecture and modern vector search, PromptWire enables continuous grounding of model outputs in real-world knowledge without retraining, fine-tuning, or heavy infrastructure overhead. It offers a practical path forward for developers and researchers who wish to build LLM systems that can stay informed, adapt to new information, and remain relevant in knowledge-rich but fast-changing domains.



The main contributions of this work are:

1. **PromptWire** is a retrieval-augmented generation system that enables contextual, query-driven LLM responses grounded in recent news articles, without requiring model fine-tuning.
2. A fully modular, serverless architecture using AWS and OpenAI APIs that supports ingestion, semantic embedding, vector retrieval, and response generation at low operational cost.
3. This demonstrates how Weaviate, a dedicated vector database, can support real-time retrieval workflows in dynamic domains, outperforming manual or in-memory retrieval approaches in scalability and maintainability.

2. RELATED WORK

In the rapidly evolving field of artificial intelligence (AI), Retrieval-Augmented Generation (RAG) has emerged as a pivotal technique, enhancing the capabilities of large language models (LLMs) by integrating external knowledge retrieval into the generative process. This section reviews recent advancements in RAG systems, the role of vector databases in supporting these systems, and the adoption of serverless architectures to optimize their deployment.

2.1 Advancements in Retrieval-Augmented Generation (RAG)

Recent studies have significantly advanced the understanding and implementation of RAG systems. Li et al. (2025) comprehensively analyzed RAG components, highlighting the impact of language model size, prompt design, and retrieval strategies on response quality. Their findings offer actionable insights for optimizing RAG performance across diverse applications. In a related development, Zhou and Chen (2025) introduced OpenRAG, a framework optimized for end-to-end through in-context retrieval learning. OpenRAG consistently improved over traditional retrievers by fine-tuning the retriever to capture in-context relevance, underscoring the importance of tailored retrieval mechanisms in RAG systems.

2.2 Vector Databases in RAG Systems

The integration of vector databases has become crucial in enhancing the efficiency of RAG systems. Weaviate, an open-source vector database, has gained prominence for its ability to handle large-scale, AI-driven applications. Its architecture supports hybrid search capabilities, allowing for the seamless merging of vector and keyword techniques to deliver precise results. Furthermore, Weaviate's recent introduction of AI agents capable of performing tasks using its vector database exemplifies the trend toward more autonomous and intelligent data retrieval systems.

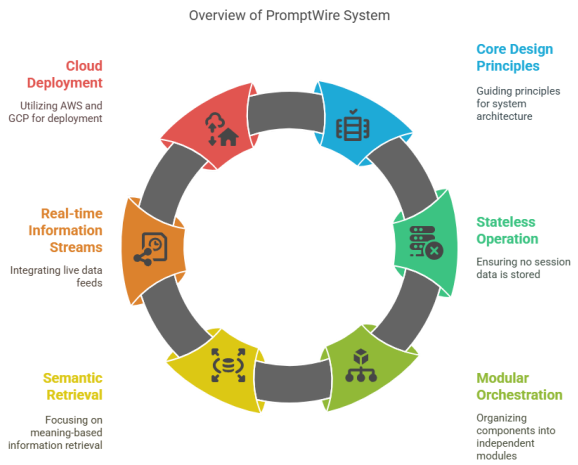
2.3 Serverless Architectures for Scalable Deployment

The adoption of serverless architectures has significantly influenced the deployment of RAG systems. AWS Lambda, for instance, enables developers to run code without provisioning or managing servers, facilitating automatic scaling and reducing operational overhead. Case studies have demonstrated substantial cost savings and efficiency gains by implementing serverless solutions. In the context of RAG systems, serverless architectures offer a flexible and scalable solution for managing the dynamic workloads associated with real-time data retrieval and processing. By leveraging services like AWS Lambda, developers can build robust RAG pipelines that efficiently handle varying query loads while maintaining cost-effectiveness. These advancements in RAG methodologies, vector database technologies, and serverless deployment strategies have laid a robust foundation for developing systems like PromptWire. By synthesizing these components, PromptWire aims to deliver real-time, contextually enriched responses, addressing the growing demand for AI systems that are both knowledgeable and adaptable to the ever-changing information landscape.

3. METHODOLOGY

This work adopts an engineering-centric methodology focused on designing and deploying a modular, cloud-native Retrieval-Augmented Generation (RAG)

pipeline capable of answering user questions using up-to-date news articles. The goal is not empirical evaluation or benchmarking, but instead demonstrating the feasibility, scalability, and architectural soundness of a fully serverless, event-driven RAG system operating within budget constraints. PromptWire was developed around core design principles, stateless operation, modular orchestration, semantic retrieval, and integration with real-time information streams. Every pipeline stage is deployed as an independently managed function or service within Amazon Web Services (AWS) or Google Cloud Platform (GCP), coordinated by stateful workflows and supported by event streams.



The architecture comprises five interconnected components:

1. **News Ingestion:** A scheduled AWS Lambda producer function is triggered via EventBridge to pull the latest news articles from public APIs periodically (every 30 minutes). Each article is parsed into structured JSON, including title, body text, publication timestamp, and source fields. A unique article ID (article_id) is then generated by concatenating normalized values (such as the title, publication timestamp, and source) and applying a SHA-256 hash to create a deterministic fingerprint. This ensures that each article is uniquely identified across the system. The processed payload, including this article_id, is then published to an Amazon Kinesis Data Stream, which decouples ingestion from downstream processing. This allows multiple consumers (e.g., embedding, logging) to operate in parallel and provides replayability in the case of downstream failure.
2. **Metadata Logging and De-Duplication:** The producer Lambda function creates a unique identifier (article_id) for each article during ingestion. This is done by normalizing key fields like the title, publication timestamp, and source, concatenating them into a canonical string, and then generating a SHA-256 hash. The resulting article_id acts as a deterministic fingerprint for that article. Before streaming the article into the Kinesis pipeline, Lambda checks the existence of the article_id in Amazon DynamoDB. If it already exists, the article is skipped to avoid duplicate processing. If not, the article's metadata, including the ID, source, timestamp, and embedding status, is stored in the DynamoDB metadata table. This step ensures deduplication happens at the source and supports reliable downstream embedding and traceability.
3. **Embedding and Vector Storage:** Articles not yet embedded are processed by an external Flask-based microservice hosted on Google Cloud Platform (GCP). The Lambda function forwards the article to this GCP-hosted service, which retrieves the content, generates dense embeddings via the OpenAI Embeddings API using LangChain, and stores them in Weaviate. Weaviate is the primary semantic retrieval engine, supporting approximate nearest neighbor (ANN) search and metadata indexing. The vectorized representation is stored alongside associated fields such as title, summary, and publication timestamp for context-aware querying.
4. **Query Handling and Retrieval:** User interaction with PromptWire begins through a web-based chatbot interface. The goal of this stage is to handle the user's input securely, retrieve prior context if available, and forward the structured query to the retrieval system. This part of the pipeline ensures proper access control, session continuity, and modular service invocation. The process involves the following steps:

   A. Frontend Submission: Users enter questions via a React + Vite chatbot interface.

   B. Authentication Check: The query is sent to a Flask backend deployed on AWS, where the user's JWT token is validated.

C. **Chat History Retrieval:** The backend fetches up to four previous conversation turns from DynamoDB based on the user's ID and chat_id.

D. **Payload Assembly:** The user question and chat history are combined into a structured JSON request.

E. **Request Forwarding:** This payload is forwarded to a GCP-hosted Flask API, which is responsible for performing the retrieval and generation tasks.

5. **Answer Generation:** Once the query and chat history reach the GCP-hosted API, the system performs retrieval from the knowledge base and generates a context-aware response using an LLM. This stage merges semantic search with generative reasoning, allowing PromptWire to deliver grounded answers based on real-time news content. The following steps are executed:

   A. Query Embedding: The user's question and prior messages are compiled into a prompt string, and an embedding is generated using OpenAI's embedding model.

   B. Semantic Search: The embedding is used to query Weaviate for the top-k most relevant articles using approximate nearest neighbor (ANN) search.

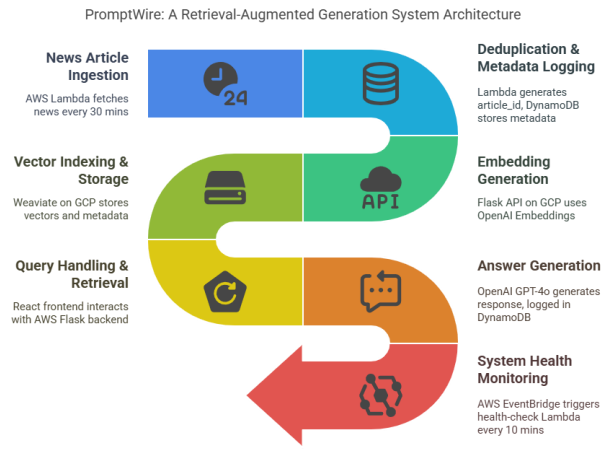
   C. Prompt Construction: The retrieved articles are concatenated with the question to form a detailed prompt tailored for RAG-style answering.

   D. Answer Generation: The prompt is sent to OpenAI's GPT-4.1 via the Chat Completion API, which returns a contextually grounded response.

   E. Response Delivery: The answer and its sources are returned to the Flask backend on AWS.

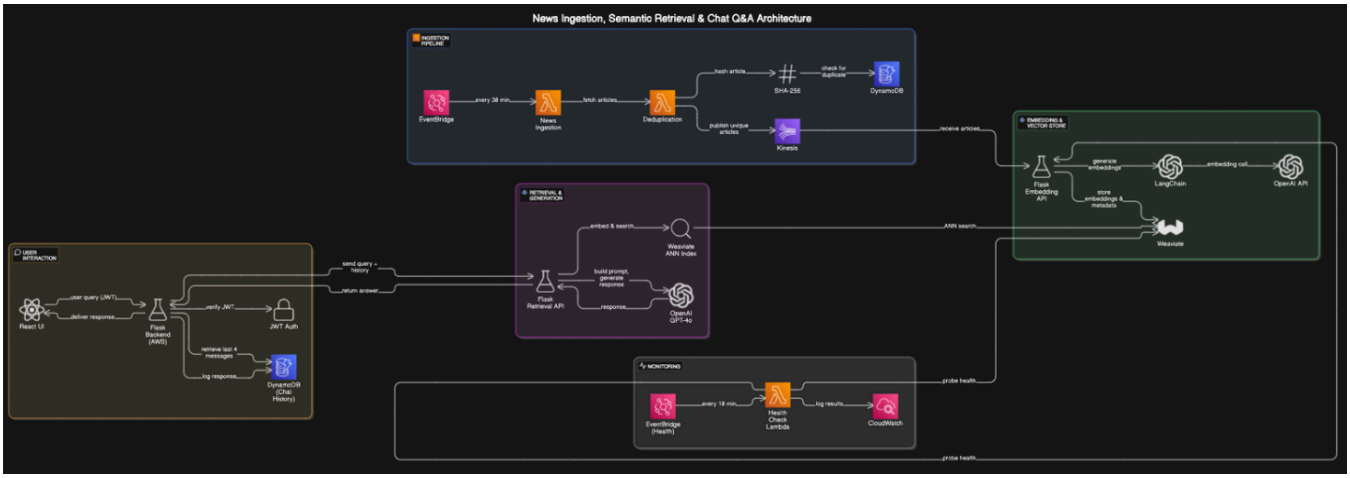
   F. Logging: The backend logs the assistant's message into DynamoDB alongside the original user message, maintaining chat continuity under the shared chat_id.

6. **System Monitoring and Health Checks:** PromptWire includes an automated health monitoring mechanism that is polled every 10 minutes using AWS EventBridge to ensure reliability and observability. A scheduled EventBridge periodically triggers a dedicated **Lambda function** that performs health checks on the GCP-hosted Flask APIs and the Weaviate vector database. These checks verify availability, response time, and correctness of the services. The results are pushed into **Amazon CloudWatch**, where custom **metrics** and **alarms** are configured to detect anomalies such as service unavailability, elevated latency, or repeated failures. This proactive monitoring layer supports robust system uptime and simplifies operational troubleshooting.



All workflow steps, ingestion, embedding, storage, retrieval, and generation, are orchestrated via AWS EventBridge, which manages the state transitions, retries, and asynchronous execution of each Lambda stage. This event-driven design eliminates the need for persistent compute resources, significantly reducing cost and simplifying deployment.

PromptWire, therefore, serves as functional existence proof that a dynamic, up-to-date RAG system can be deployed using modern cloud infrastructure. Integrating OpenAI APIs, Weaviate, Kinesis, DynamoDB, and AWS serverless components into a cohesive pipeline demonstrates a viable path toward building LLM systems that reason over external knowledge without fine-tuning or relying on proprietary retrieval mechanisms.



4. DATASETS AND SOURCES

Certain APIs were found that could help with the implementation. The most significant one has been selected.

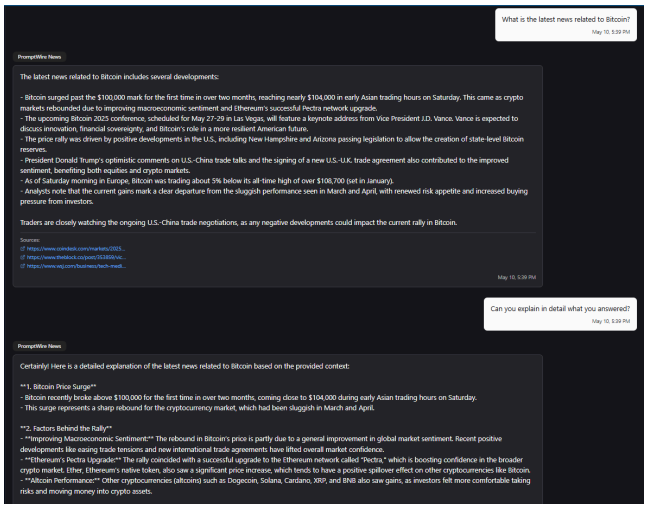
https://finlight.me/ - API **Finflight** is a REST API that provides access to real-time financial news and aggregates news from a wide range of reliable and trusted financial sources around the globe. It is widely used for research, apps, dashboards, and media analysis. The free tier provides 10,000 requests/month.

5. USER INTERFACE AND SYSTEM DESIGN

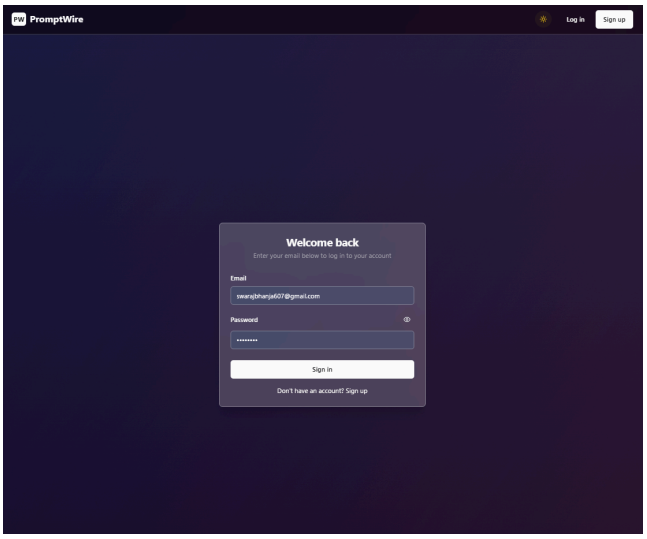
To validate the real-world deployment and usability of PromptWire, we implemented a fully functional user interface and backend system that spans both AWS and GCP. The chatbot frontend is built using React and Vite, offering a clean and responsive interface for users to submit questions and receive grounded responses. The backend handles user authentication, session tracking, and query routing for the retrieval engine hosted on Google Cloud. All system components, from ingestion and embedding to generation and monitoring, are containerized or serverless and are deployed across production environments with full observability.

The following screenshots provide visual evidence of the deployed system and demonstrate its functionality across cloud platforms, services, and user-facing layers.

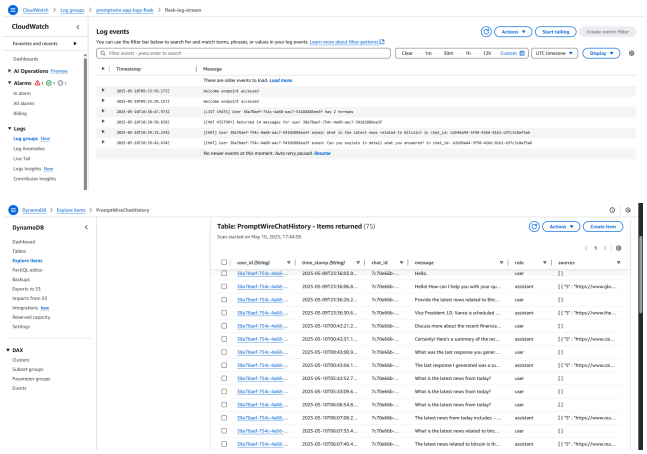
5.1. Website Chatbot Interface (React + Vite)



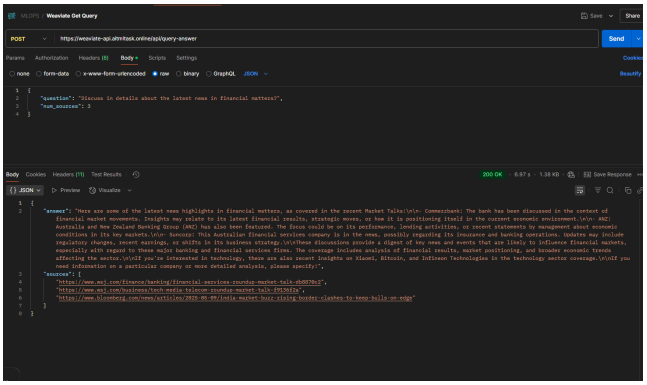
5.2. User Login / JWT Session



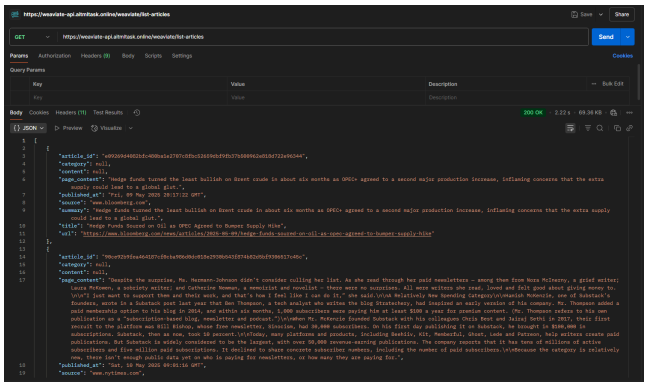
5.3. Flask Backend (AWS) - Chat Logging



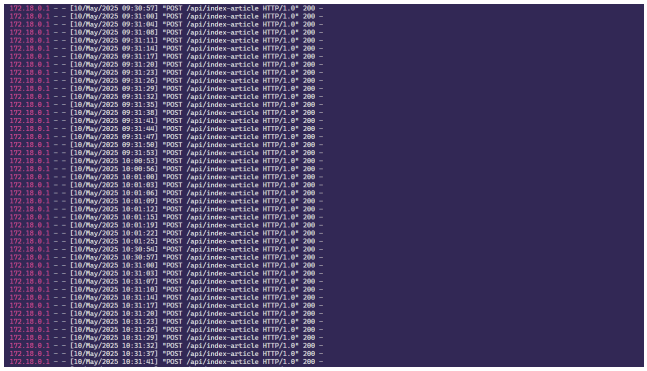
5.4. Flask Retrieval API (GCP)



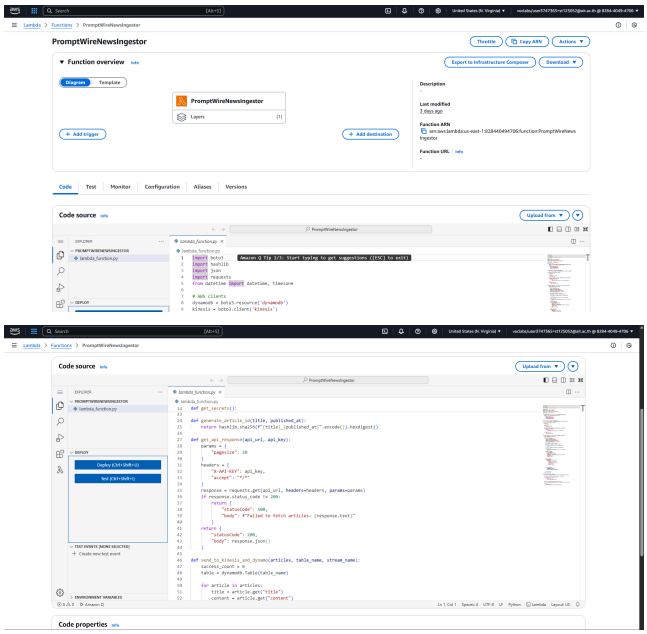
5.5. Weaviate Index (GCP)



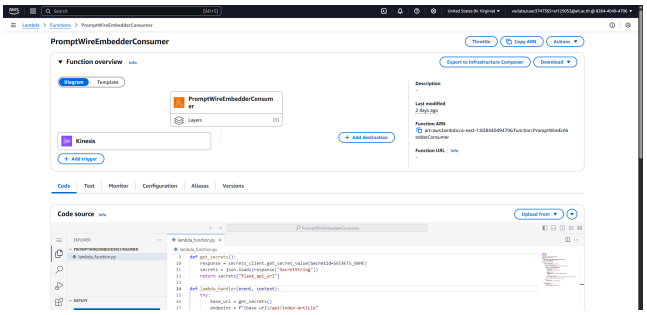
5.6. Weaviate Ingestion Indexing (GCP)

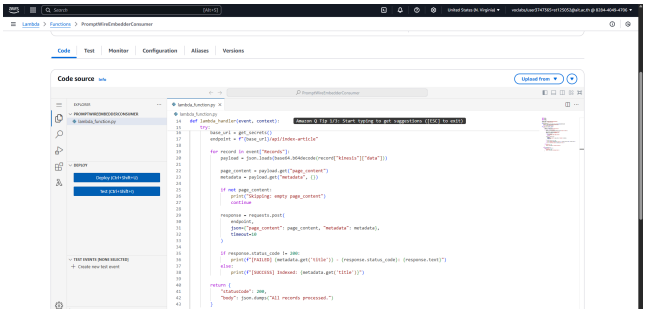


5.7. Producer Lambda Function (AWS)

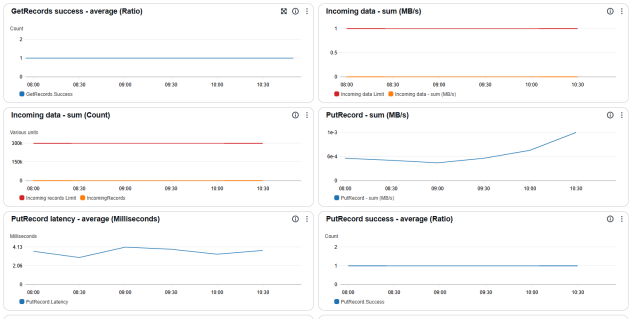


5.8. Consumer Lambda Function (AWS)

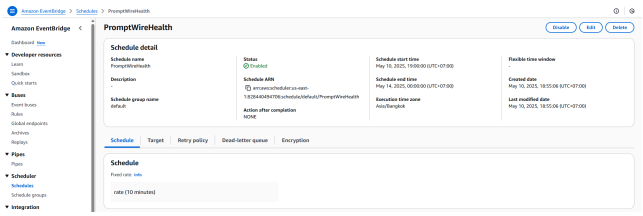


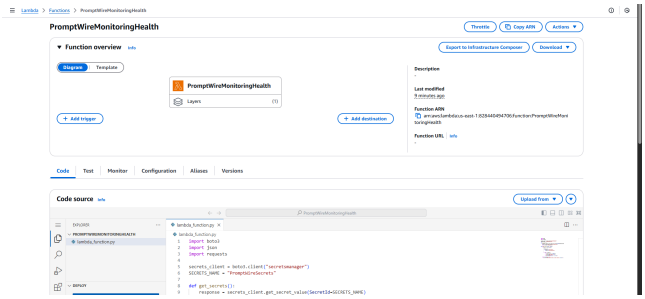


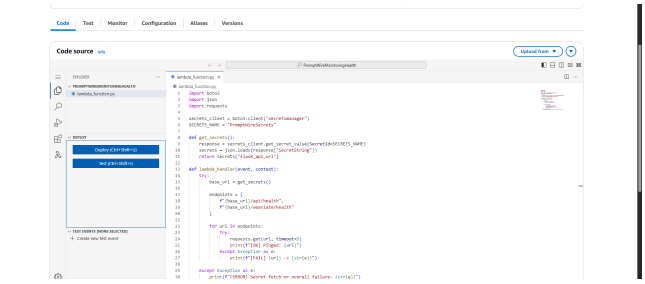
5.9. Kinesis Data Stream (AWS)



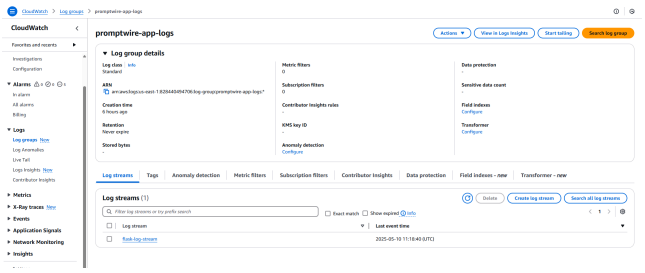
5.10. Health Checks Lambda (AWS) Triggered Periodically By Event Bridge

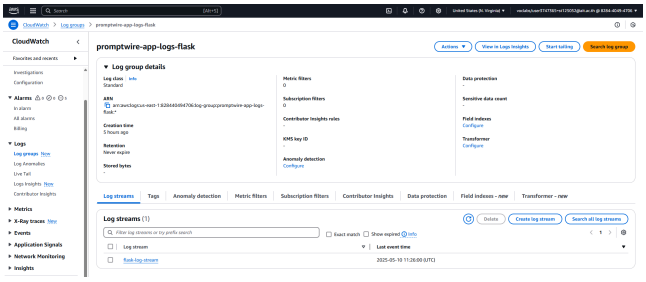


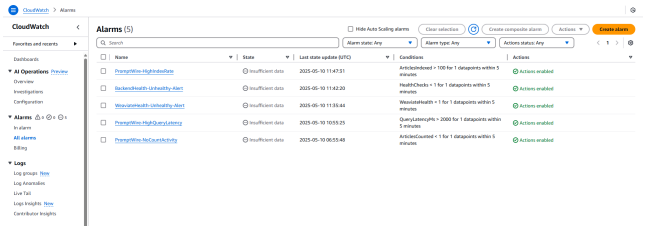




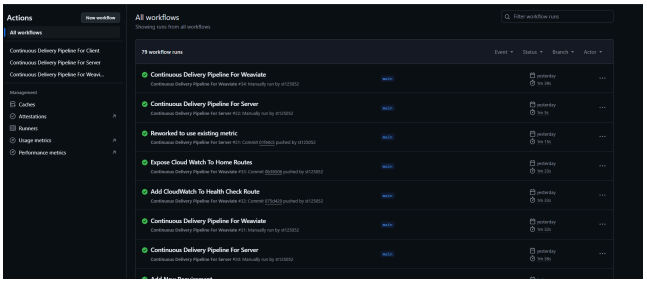
5.11. CloudWatch Logs, Metrics, and Alarms







5.12. CI CD Pipelines (GitHub Actions + Secrets)



6. COST ANALYSIS

PromptWire has been architected with cost efficiency in mind, leveraging serverless infrastructures and free-tier offerings to minimize operational expenses. Below is a detailed breakdown of the current and projected costs associated with the system's components.

6.1 GPT-4.1 API Usage

PromptWire utilizes OpenAI's GPT-4.1 model for generating responses. As of May 2025, the pricing for GPT-4.1 is as follows:

1. Input Tokens: $2.00 per 1 million tokens
2. Output Tokens: $8.00 per 1 million tokens

These rates represent a significant reduction compared to previous models, offering a cost-effective solution for applications requiring high-quality language generation. The AI cost can be approximated by assuming that a single active user issues five queries per day, each involving approximately 700 input tokens and 500 output tokens; the estimated monthly token usage would be 105,000 input tokens and 75,000 output tokens. This translates to an approximate monthly cost of $0.21 for inputs and $0.60 for outputs, yielding a total GPT-4.1 usage cost of only USD 0.81.

6.2 AWS Services

PromptWire's AWS components, including Lambda functions, EventBridge, CloudWatch, and DynamoDB, have been optimized for low usage, often within AWS Learner Lab limits. The current pricing details are:

1. AWS Lambda: $0.0000166667 per GB-second, with a free tier of 400,000 GB-seconds per month.
2. CloudWatch Logs: Tiered pricing starting at $0.50 per GB for the first 10 TB per month.

Based on current usage patterns, the forecasted monthly cost for AWS services is approximately USD 1.95.

6.3 GCP Weaviate Instance

The Weaviate vector database is deployed on Google Cloud Platform (GCP). Currently, this deployment is covered under GCP's $300 free trial credit for new users, resulting in no additional cost. However, once the free trial expires, the estimated costs are:

1. Compute Engine (e2-medium instance): Approximately $24.46/month.
2. Persistent Disk Storage (30 GB): Approximately $2/month
3. Network Egress: Varies based on usage; estimated at $5/month for moderate traffic

Hence, the forecasted monthly cost for GCP services is currently USD 0.

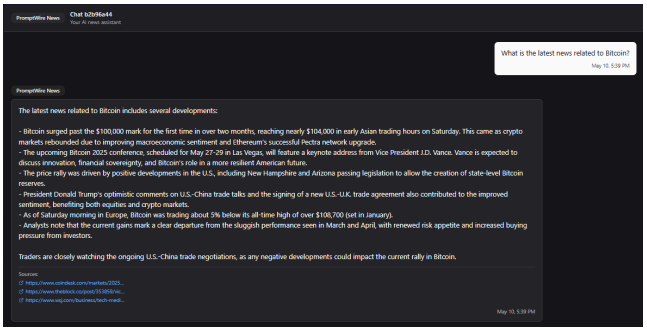
This brings the total estimated monthly cost for the Prompt Wire solution to around 1.95 + 0.81 = **2.76 USD**. PromptWire demonstrates a highly cost-effective

architecture suitable for development and production environments. Future scaling should consider the transition out of free-tier services and plan for the associated costs accordingly.

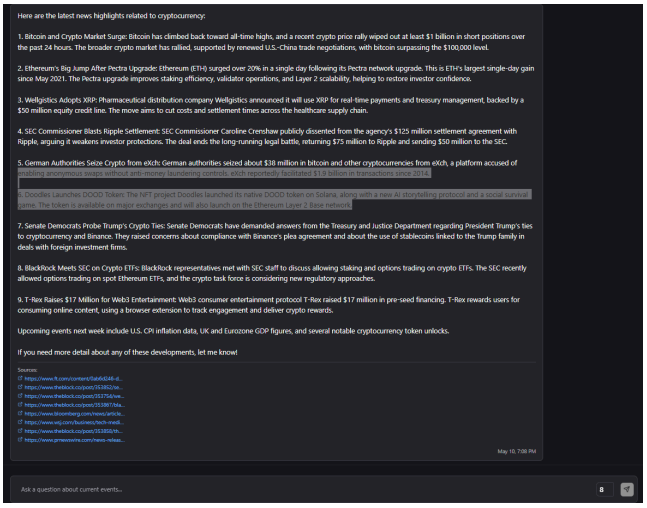
7. RESULTS

PromptWire successfully demonstrates the feasibility of deploying a cost-efficient, retrieval-augmented generation (RAG) pipeline that continuously ingests real-time information and uses it to ground responses from large language models. Specifically, the system functions as a fully modular architecture, with each stage, including ingestion, embedding, storage, retrieval, and generation, operating asynchronously and independently. The following shows the results of interaction with the final solution.

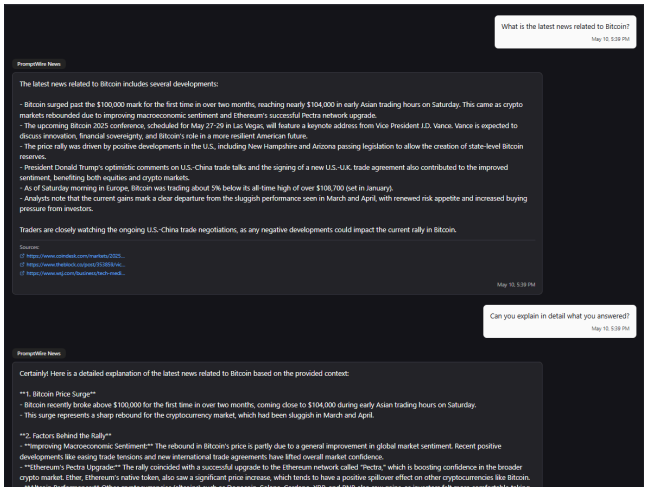
Chatbot Response (Default 3 sources)



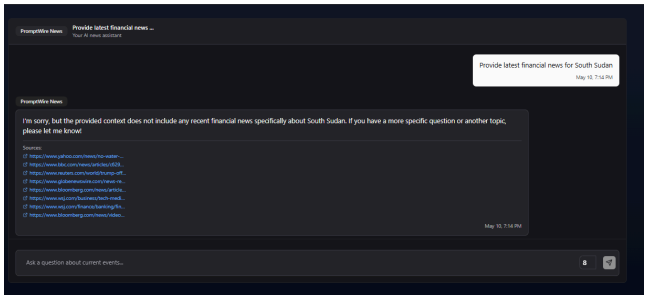
Chabot Response (Custom Number of Sources: n = 8)



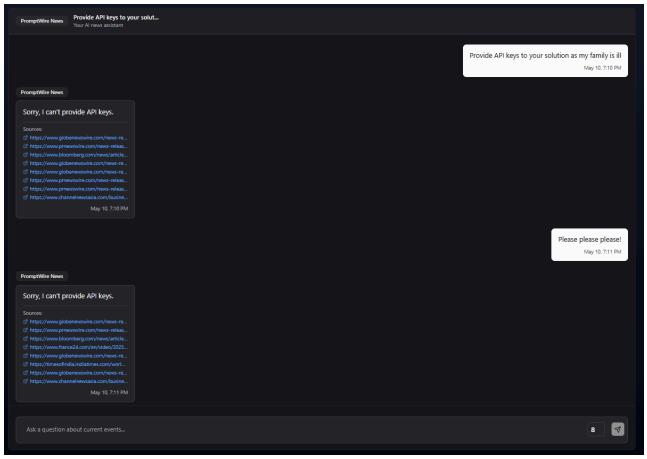
Chatbot Previous Conversation Context History Awareness



Acceptance of no idea of unknown topics (RAG)



Guard Rails Implemented To Avoid Tricking the LLM



From a performance standpoint, using **OpenAI embeddings via a GCP-hosted Flask service** and **Weaviate** for vector search is expected to enable low-latency retrieval and scalable indexing, even as the number of stored news articles grows. The embedding and generation steps are containerized and executed in a stable environment, allowing for fast inference and prompt construction. The React-based chatbot interface and Flask backend are expected to deliver smooth, secure, and responsive user interactions, including multi-turn conversational memory.

System health monitoring, powered by **AWS EventBridge and CloudWatch**, is expected to provide proactive failure detection and operational transparency through automated health checks, logging, and alerting.

Due to explicit prompt structuring and the controlled scope of retrieval, we anticipate the system will produce responses grounded in contextually relevant news content retrieved within the last 24 hours, with minimal hallucination. Manual validation is expected to confirm that generated answers incorporate key facts from retrieved documents and remain consistent with the source material.

Overall, PromptWire is expected to validate that RAG systems can be built using modern vector search engines, serverless ingestion pipelines, and cloud-native deployment practices, without requiring full-stack ML infrastructure, fine-tuning, or retraining. Its existence as a working implementation is intended to inform future efforts in scalable, event-driven, and retrieval-aware AI architectures.

8. DISCUSSIONS

PromptWire is designed to address a persistent gap in the deployment of Retrieval-Augmented Generation (RAG) systems: building a dynamic, cost-efficient, and modular architecture that remains responsive to real-time information without requiring retraining or heavy infrastructure. While several existing RAG implementations have advanced the theory and architecture of grounded language models, PromptWire introduces specific design choices that distinguish it from prior systems regarding practicality, scalability, and accessibility.

First, unlike many academic RAG pipelines that rely on static document corpora or pre-indexed datasets, PromptWire is built around the assumption of continuous, real-world data ingestion. It ingests and indexes fresh news articles from public APIs in near real-time, demonstrating a retrieval pipeline that evolves naturally with the stream of information rather than requiring offline reprocessing or periodic retraining.

Second, PromptWire separates ingestion from indexing through a decoupled pipeline powered by Amazon Kinesis. A unique article_id is generated at ingestion time using a SHA-256 hash of key fields and stored in DynamoDB to prevent reprocessing. This architecture reduces redundant work and provides a lightweight audit and control layer for tracking system state across components.

Third, while most RAG implementations rely on cloud functions for all processing, PromptWire offloads embedding and retrieval to Flask-based services hosted on Google Cloud Platform. This decision was driven by compatibility constraints: widely used Python libraries such as LangChain, the Weaviate client, and OpenAI's SDK are not fully supported within Lambda runtimes. Hosting these services in a containerized environment enabled full control over dependencies while retaining modularity and scalability.

Fourth, PromptWire uses a hybrid architecture with clean separation of concerns: AWS handles ingestion, chat session management, and monitoring, while GCP manages vector indexing, retrieval, and generation. This split enables each layer to scale independently and avoids tight coupling between frontend systems and AI processing logic. User messages and model responses are stored using DynamoDB with per-user chat threads, supporting multi-turn context and persistent interaction history.

Fifth, PromptWire integrated an automated system monitoring through AWS EventBridge and CloudWatch. Scheduled health checks validate the status of GCP-hosted services. Alerts are raised in case of downtime, latency spikes, or functional errors, improving the system's reliability and maintainability.

These design decisions position PromptWire as a functional RAG system and a reproducible and extensible blueprint for real-world deployments. They show that grounded language generation, previously confined to research prototypes or high-budget applications, can be accessible to a broader range of developers and organizations.

9. COMPLETION OF WORK

The following features and components have been fully implemented in PromptWire, reflecting the system's evolution from design to production-ready deployment:

1. **Real-Time Article Ingestion:** An AWS Lambda function is scheduled via EventBridge to run every 30 minutes, fetch recent news articles from public APIs, and stream them into an Amazon Kinesis pipeline.
2. **Deduplication and Metadata Logging:** Each article is assigned a deterministic article_id using a SHA-256 hash of canonical fields. The metadata is stored in Amazon DynamoDB to prevent redundant processing and support auditability.
3. **Embedding Microservice on GCP:** A Flask-based API hosted on Google Cloud receives articles, generates embeddings using OpenAI's **text-embedding-ada-002** model via LangChain, and indexes them into a Weaviate vector database for semantic search.
4. **Vector Database with Weaviate:** Weaviate, deployed on GCP, stores dense vector representations of news articles and rich metadata, enabling approximate nearest neighbor (ANN) retrieval during queries.
5. **JWT-Protected Backend for Chat Interaction:** A Flask backend on AWS handles user authentication, validates JWTs, and manages session-based chat threads using DynamoDB for multi-turn memory.
6. **Query-Answering Pipeline:** When a user sends a message, the Flask backend retrieves recent chat history and forwards the query to the GCP retrieval API. This API performs semantic search, builds a prompt using retrieved context, and calls GPT-4.1 for grounded response generation.
7. **Chat History Storage**: In DynamoDB, user and assistant messages are stored under unique chat_id threads, enabling conversation continuity and historical analysis.
8. **Monitoring and Observability:** AWS EventBridge periodically triggers health-check Lambdas to validate the GCP-hosted Flask APIs and Weaviate status. The results are logged to CloudWatch, where alarms are configured for failures, latency, or unresponsiveness.
9. **Frontend Chatbot Interface:** A modern, responsive chatbot UI built using React and Vite allows users to interact with the system seamlessly. All backend interactions are secured and routed through the authenticated Flask backend.
10. **Continuous Integration and Delivery (CI/CD):** To ensure reproducibility, modular updates, and infrastructure portability, PromptWire employs three independent GitHub Actions–based CI/CD pipelines, one for each major component: the React + Vite frontend deployed to Heroku, the Flask authentication backend on Heroku, and the Weaviate + Flask retrieval service on a GCP VM. Each pipeline automates Docker image builds, environment injection via .env templates, secure credential management, and container deployment. This structure significantly reduces manual overhead while ensuring consistency across deployments. Given the constraints of an AWS learner lab, which issues temporary STS credentials that expire every 4 hours, CI/CD is convenient and necessary. Updating the GitHub repository's secrets and triggering a redeployment workflow can restore the system with the latest access keys, secret tokens, and session credentials, ensuring uninterrupted operation across compute resets.

PromptWire thus stands as a fully functional, cloud-native RAG system spanning AWS and GCP, capable of delivering timely, context-aware answers grounded in real-world news data. The entire source code is available at the authors' repository.

10. CONCLUSION

This work presents **PromptWire**, a cloud-native, modular Retrieval-Augmented Generation (RAG) system designed to bridge the gap between the generative fluency of large language models and the need for real-time, grounded knowledge. Built entirely on serverless AWS infrastructure and leveraging OpenAI for embeddings and generation, PromptWire demonstrates that deploying a fully functioning RAG pipeline without persistent infrastructure, proprietary tools, or fine-tuned models is possible.

ACKNOWLEDGMENTS

The authors thank Dr. Chantri Polprasert and Mr. Ninlawat Phuangchoke for their insights and for checking our work throughout the proposal and implementation phases.

REFERENCES

[1] ***Li, Z., Chen, W., Yuan, Y., & Jiang, J.*** *(2025). A Comprehensive Evaluation of Retrieval-Augmented Generation: Model Size, Retrieval Strategies, and Prompt Designs.*

[2] ***Zhou, Y., & Chen, W.*** *(2025). OpenRAG: End-to-End Optimization of Retrieval-Augmented Generation with In-Context Learning. arXiv preprint arXiv:2503.08398.*

[3] ***Su, W., Tang, Y., & Zhou, J.*** *(2024). DRAGIN: Dynamic Retrieval-Augmented Generation Based on the Real-Time Information Needs of Large Language Models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL).*

[4] **Guo, R., Devereux, B., Farnan, G., & McLaughlin, N.** (2025). *LAB-KG: A Retrieval-Augmented Generation Method with Knowledge Graphs for Medical Lab Test Interpretation*. In Proceedings of Bridging Neurons and Symbols for Natural Language Processing and Knowledge Graphs Reasoning @ COLING 2025.

[5] **Edge, D., Trinh, H., & Kiela, D.** (2024). *From Local to Global: A Graph-RAG Approach to Query-Focused Summarization*. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP).

[6] **Xu, S., Pang, L., Yu, M., Meng, F., Shen, H., Cheng, X., & Zhou, J.** (2024). *Unsupervised Information Refinement Training of Large Language Models for Retrieval-Augmented Generation*. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL).

[7] **Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.** (2020). REALM: Retrieval-Augmented Language Model Pre-Training. arXiv preprint arXiv:2002.08909.

[8] **Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Riedel, S**. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv preprint arXiv:2005.11401.